

Sétima Lista de Engenharia de Software RT

Capítulos 7 e 8 — Projeto, Implementação e Testes de Software

Ian Sommerville — *Software Engineering*, 10^a ed. + Exercícios IAS — Universität Stuttgart
(Exercises 10 e 11)

Aluno: Matheus Serrão Uchoa

Esta lista reúne quatro partes: Parte I — questões do Capítulo 7 (projeto e implementação); Parte II — questões do Capítulo 8 (testes de software); Parte III — *Exercise* 10 (implementação de *state charts*, robô de limpeza, programação estruturada); Parte IV — *Exercise* 11 (testes: objeto/driver/caso de teste e medição de cobertura), do material do IAS da Universität Stuttgart.

Sumário

1	Parte I — Capítulo 7 (Sommerville): Projeto e Implementação	1
2	Parte II — Capítulo 8 (Sommerville): Testes de Software	7
3	Parte III — Exercise 10 (Stuttgart): Implementação	12
4	Parte IV — Exercise 11 (Stuttgart): Teste	15

1 Parte I — Capítulo 7 (Sommerville): Projeto e Implementação

Questão 1 — Casos de uso da estação meteorológica

Enunciado

Usando a notação estruturada do Quadro 7.1, especifique os casos de uso da estação meteorológica para *Relatar status* e *Reconfigurar*. Faça suposições razoáveis sobre a funcionalidade necessária.

Resposta

Sistema: Estação meteorológica Caso de uso: Relatar status	
Atores	Sistema de informações meteorológicas.
Dados	A estação devolve um relatório de status com o estado de hardware/-software de cada instrumento, o nível de carga da bateria e a data/hora da última coleta.
Estímulo	Pedido de status enviado pelo sistema de informações meteorológicas.
Resposta	Relatório de status transmitido ao solicitante.
Comentários	O status costuma ser solicitado periodicamente (p. ex., diariamente) para monitorar a saúde da estação remota.

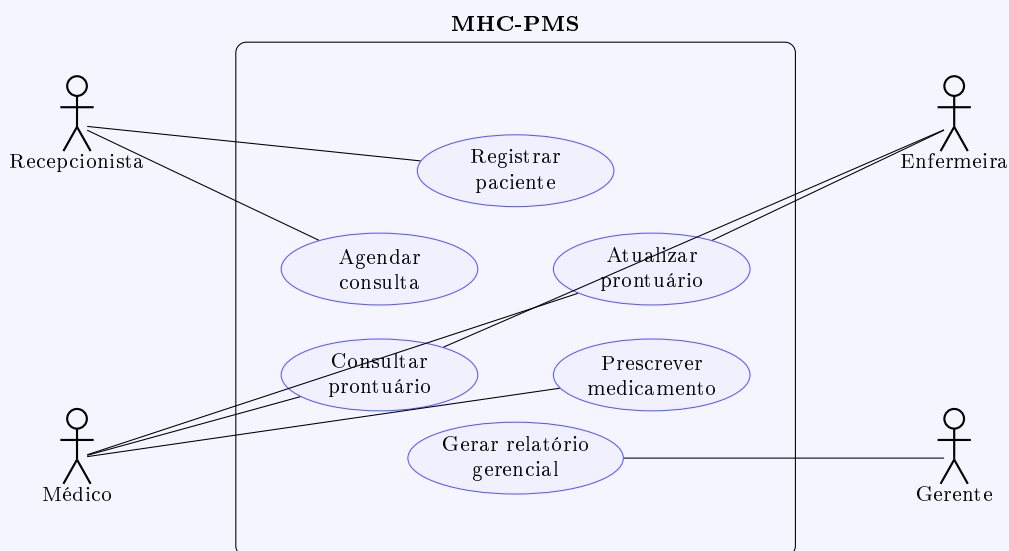
Sistema:	Estação meteorológica	Caso de uso:	Reconfigurar
Atores	Sistema de informações meteorológicas (controle).		
Dados	Comandos de configuração e/ou novo software a ser instalado (parâmetros de coleta, frequência de amostragem, atualização de firmware).		
Estímulo	Comando de reconfiguração enviado pelo sistema de informações meteorológicas.		
Resposta	Confirmação da reconfiguração; a nova configuração passa a vigorar.		
Comentários	A estação deve entrar em modo de manutenção/espera durante a operação; em caso de falha, deve reverter para a configuração anterior (<i>rollback</i>).		

Questão 2 — Diagrama de casos de uso do MHC-PMS

Enunciado

Supondo que o MHC-PMS seja desenvolvido por uma abordagem orientada a objetos, desenhe um diagrama de casos de uso mostrando pelo menos seis possibilidades para esse sistema.

Resposta



São seis casos de uso: registrar paciente, agendar consulta, consultar prontuário, atualizar prontuário, prescrever medicamento e gerar relatório gerencial, com os papéis (atores) que os dispáram.

Questão 3 — Classes de objeto (atributos e operações)

Enunciado

Projete as seguintes classes de objeto, identificando atributos e operações: um telefone; uma impressora para PC; um sistema de som estéreo pessoal; uma conta bancária; um catálogo de biblioteca.

Resposta

Telefone
- número
- agenda
- emChamada : bool
+ discar(n)
+ atender()
+ desligar()
+ salvarContato()

Impressora
- modelo
- nívelTinta
- fila : Doc[*]
- status
+ imprimir(doc)
+ cancelar(doc)
+ statusTinta()

SomEstéreo
- volume
- fonte
- estação
- ligado : bool
+ ligar()
+ ajustarVolume(v)
+ selecionarFonte()
+ sintonizar(f)

ContaBancária
- número
- saldo
- titular
- limite
+ depositar(v)
+ sacar(v)
+ transferir(c,v)
+ extrato()

CatálogoBiblioteca
- acervo : Item[*]
- índicePorAutor
- índicePorTítulo
+ buscar(critério)
+ adicionarItem(i)
+ removerItem(i)
+ verificarDisponib.(i)

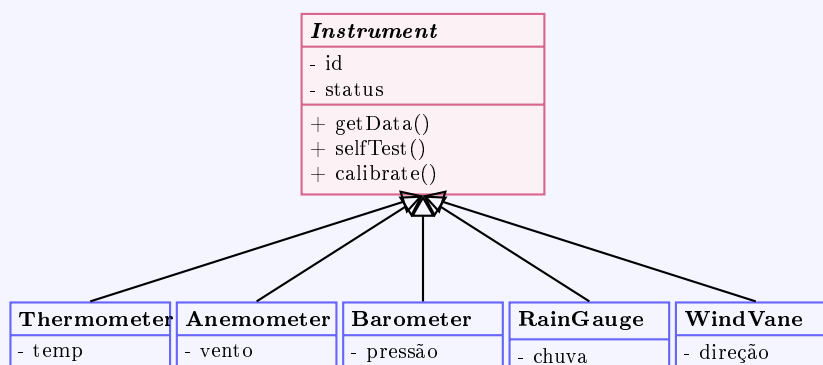
Questão 4 — Objetos da estação e hierarquia de herança

Enunciado

A partir dos objetos da estação meteorológica identificados na Figura 7.5, identifique outros objetos que possam ser usados no sistema e projete uma hierarquia de herança para os objetos identificados.

Resposta

Além de *WeatherStation* e *WeatherData* (Figura 7.5), identificam-se os **instrumentos** de medição. Todos compartilham identificador e operações de leitura e autoteste, o que motiva a superclasse abstrata *Instrument*.



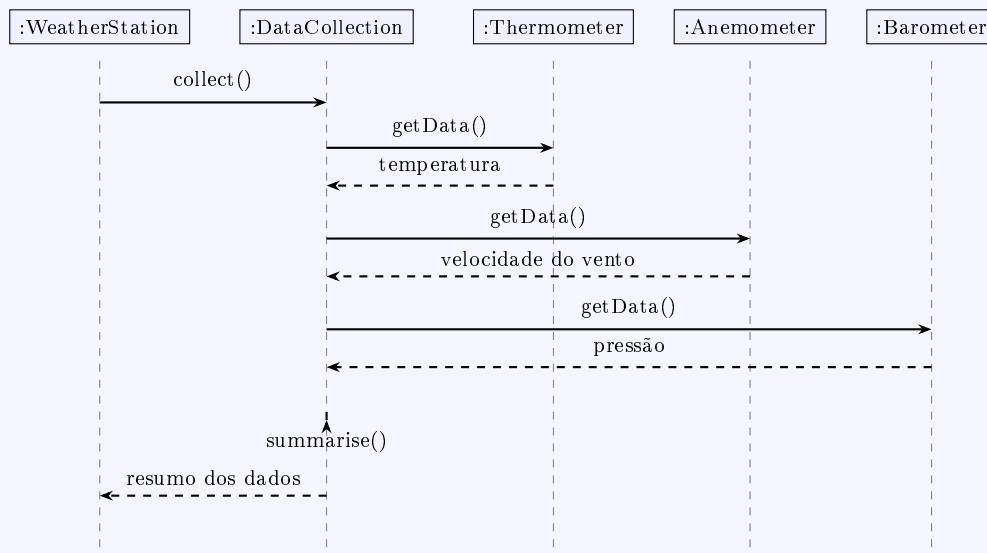
A *WeatherStation* agrega um conjunto de *Instruments* e delega a coleta a cada um polimorficamente (`getData()`); novos instrumentos podem ser adicionados sem alterar a estação.

Questão 5 — Sequência: coleta de dados e instrumentos

Enunciado

Desenvolva o projeto da estação meteorológica mostrando, com diagramas de sequência, a interação entre o subsistema de coleta de dados e os instrumentos que coletam dados meteorológicos.

Resposta



O subsistema *DataCollection* consulta sequencialmente cada instrumento (`getData()`), agrega os valores (`summarise()`) e devolve o resumo à *WeatherStation*.

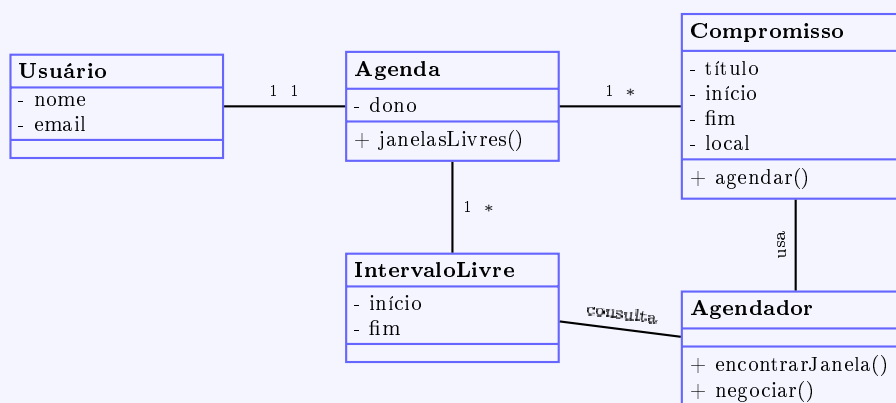
Questão 6 — Projeto OO: agenda de grupo e posto de gasolina

Enunciado

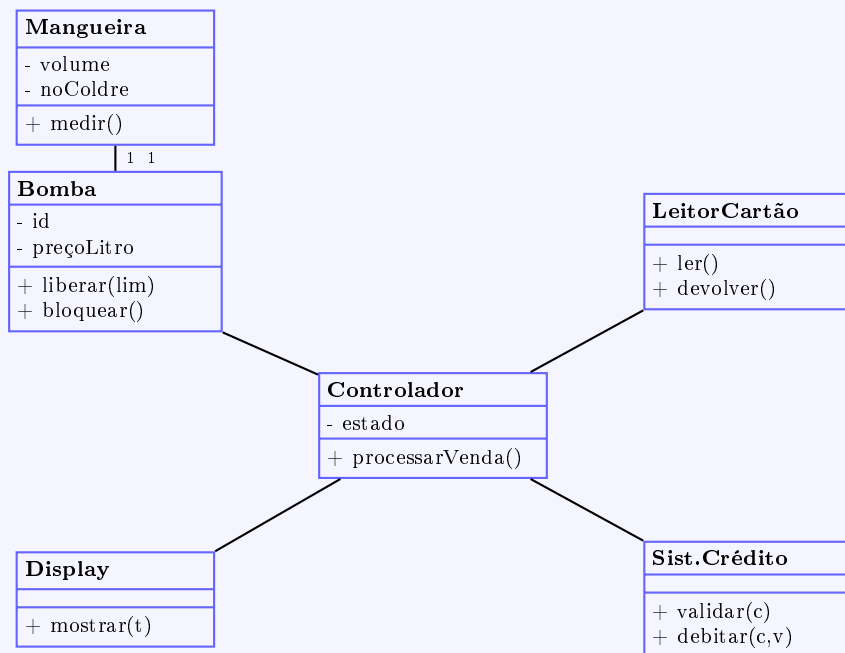
Identifique possíveis objetos e desenvolva um projeto orientado a objetos para: (a) um sistema de agenda/gerenciamento de tempo de grupo; (b) um posto de gasolina automatizado.

Resposta

(a) Sistema de agenda de grupo:



(b) Posto de gasolina automatizado:

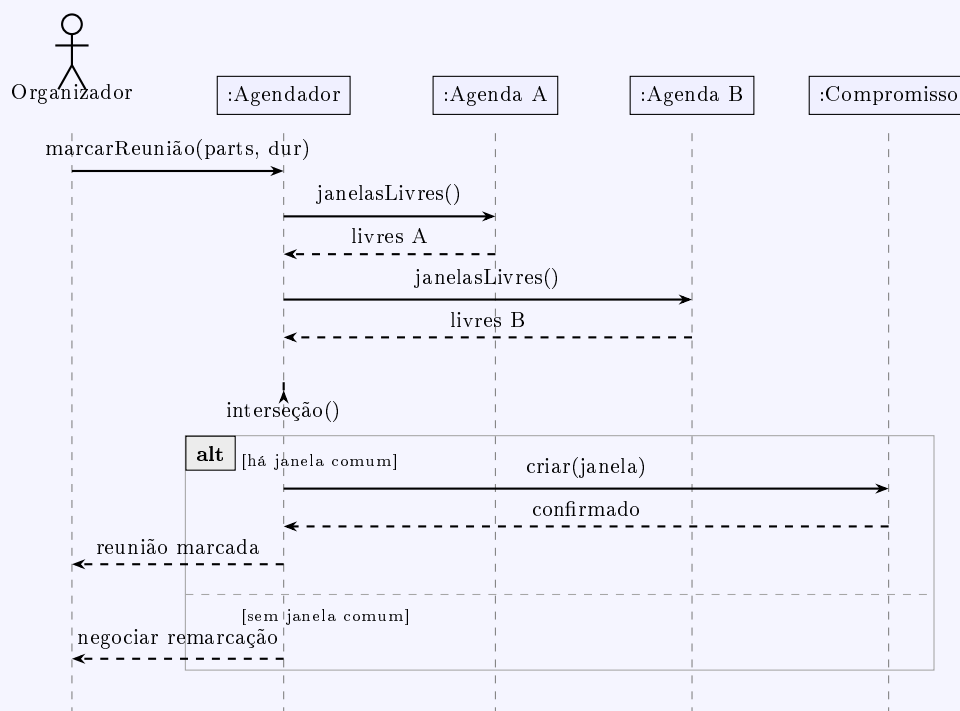


Questão 7 — Sequência: agendamento de reunião (agenda de grupo)

Enunciado

Desenhe um diagrama de sequência mostrando as interações dos objetos em um sistema de agenda de grupo quando um grupo de pessoas organiza uma reunião.

Resposta



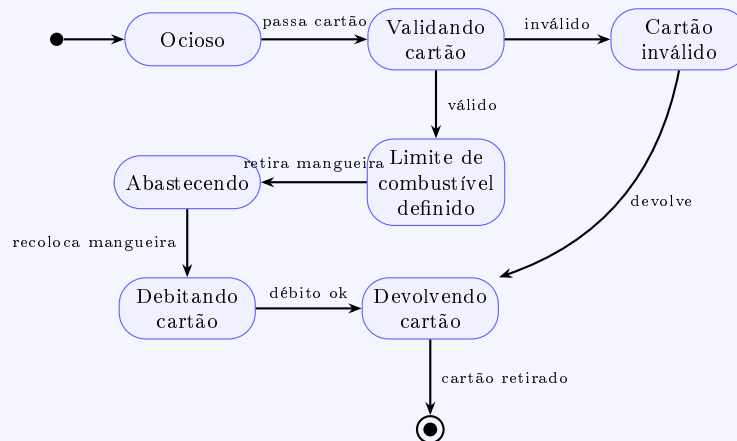
Questão 8 — Diagrama de estado (posto de gasolina)

Enunciado

Desenhe um diagrama de estado da UML mostrando as possíveis mudanças de estado na agenda de grupo ou no sistema do posto de gasolina.

Resposta

Modelando o **posto de gasolina automatizado**:



Questão 9 — Importância do gerenciamento de configuração

Enunciado

Usando exemplos, explique por que o gerenciamento de configuração é importante quando uma equipe desenvolve um produto de software.

Resposta

O gerenciamento de configuração (GC) controla a evolução de um sistema composto por muitos artefatos que mudam de forma independente e simultânea por vários desenvolvedores. Sua importância:

- **Controle de versões / evitar conflitos:** se dois desenvolvedores editam o mesmo arquivo, o GC (Git/SVN) detecta e funde as mudanças, impedindo que um sobrecreva o trabalho do outro.
- **Rastreabilidade e reversão:** é possível voltar a uma versão estável. *Exemplo:* uma release introduz um defeito grave; com o GC reverte-se ao último *commit* bom enquanto se corrige o problema.
- **Gestão de *builds* e releases:** reproduzir exatamente a versão entregue a um cliente. *Exemplo:* o cliente X usa a v2.3; para corrigir um *bug* dele, recupera-se o *baseline* 2.3.
- **Gerência de mudanças:** associa solicitações de mudança/defeitos às alterações de código, dando histórico e auditoria.
- **Trabalho paralelo com *branches*:** desenvolver uma nova funcionalidade sem desestabilizar a linha principal.

Sem GC, equipes perdem alterações, não conseguem reproduzir versões entregues e têm dificuldade de coordenar o trabalho — problemas que crescem com o tamanho da equipe.

Questão 10 — Tornar o software *open source*

Enunciado

Uma pequena empresa desenvolve um produto que se configura especialmente para cada cliente. Surge a oportunidade de um contrato que mais que dobraria sua base de clientes, e o novo cliente quer envolvimento na configuração. Explique por que, nessas circunstâncias, pode ser boa ideia tornar o software *open source*.

Resposta

- **Capacidade limitada de customização:** a empresa é pequena e pode não dar conta de configurar para uma base de clientes muito maior. Abrindo o código, clientes e terceiros assumem parte do trabalho de configuração — exatamente o que o novo cliente deseja.
- **Modelo de negócio em serviços:** o valor migra da venda de licenças para suporte, consultoria, customização e treinamento — receitas recorrentes que escalam melhor para uma base maior.
- **Contribuições da comunidade:** correções e melhorias feitas pelos usuários retornam ao produto, reduzindo o custo de manutenção.
- **Confiança e redução de *lock-in*:** o cliente grande não fica refém de um fornecedor pequeno (risco de descontinuidade); o acesso ao código o tranquiliza quanto à continuidade e à auditabilidade.
- **Adoção e padronização:** mais usuários podem transformar o produto em padrão de fato no nicho, ampliando o mercado de serviços.

Ressalvas: perde-se a receita de licenças e expõe-se o código a concorrentes; é preciso escolher bem a licença (p. ex., *copyleft* para forçar o retorno de melhorias) e separar eventuais módulos proprietários que constituam a vantagem competitiva central.

2 Parte II — Capítulo 8 (Sommerville): Testes de Software

Questão 1 — Entregar software não totalmente livre de defeitos

Enunciado

Explique por que um programa não precisa, necessariamente, ser completamente livre de defeitos antes de ser entregue a seus clientes.

Resposta

É praticamente impossível e economicamente inviável garantir ausência total de defeitos: o espaço de entradas/estados é gigantesco e testes exaustivos não são factíveis. Além disso:

- **Nem todo defeito se manifesta:** muitos só ocorrem em condições raras que talvez nunca aconteçam na operação real; o custo de removê-los supera o benefício.
- **Valor de negócio:** entregar mais cedo, mesmo com defeitos menores conhecidos, pode valer mais que atrasar à espera da perfeição (janela de mercado, contratos).
- **Criticidade variável:** defeitos cosméticos ou de baixa severidade podem ser corrigidos em atualizações posteriores.
- **Custo marginal crescente:** a “lei” dos retornos decrescentes faz a remoção dos últimos defeitos custar muito mais do que dos primeiros.

A decisão é baseada em risco: aceita-se entregar quando o nível de confiança é adequado

ao domínio (sistemas críticos exigem muito mais rigor que aplicativos comuns).

Questão 2 — Testes mostram presença, não ausência de erros

Enunciado

Explique por que os testes podem detectar apenas a presença de erros, e não a sua ausência.

Resposta

Um teste executa o programa para um *subconjunto finito* das entradas possíveis. Se um teste falha, prova-se a **existência** de pelo menos um defeito. Mas, como o conjunto de entradas/estados é praticamente infinito, passar em todos os testes selecionados não cobre todas as situações — sempre podem existir entradas não testadas que revelariam falhas. Logo, testes **nunca provam** que não há erros (frase atribuída a Dijkstra: “*testing shows the presence, not the absence of bugs*”). A confiança obtida é estatística, não uma prova de correção (que exigiria verificação formal).

Questão 3 — Desenvolvedores testarem o próprio código

Enunciado

Dê argumentos a favor e contra a realização de testes pelos próprios desenvolvedores (em vez de uma equipe independente).

Resposta

A favor:

- Conhecem o código profundamente e encontram rapidamente onde testar.
- *Feedback* imediato: corrigem na hora, reduzindo custo do defeito.
- Viabilizam testes de unidade automatizados (TDD) e CI.

Contra:

- **Viés de confirmação:** tendem a testar o que “deveria funcionar”, repetindo as mesmas suposições que geraram o defeito.
- Conflito de interesse: pressa/orgulho podem reduzir o rigor.
- Uma equipe independente traz olhar imparcial, testa requisitos (não implementação) e cobre cenários inesperados.

Conclusão: abordagem combinada — desenvolvedores fazem testes de unidade e integração; uma equipe independente faz testes de sistema, release e aceitação.

Questão 4 — Partições e testes para catWhiteSpace

Enunciado

Teste o método `catWhiteSpace` de um objeto `Parágrafo`, que substitui sequências de espaços em branco por um único espaço. Identifique partições de equivalência e derive um conjunto de testes.

Resposta

Partições de equivalência (entrada = string do parágrafo):

Partição	Comportamento esperado
String vazia	Retorna string vazia.
Sem espaços em branco	Inalterada.
Espaços simples isolados (já normalizada)	Inalterada.
Sequência de múltiplos espaços	Cada sequência vira 1 espaço.
Outros brancos (tab \t, \n)	Tratados como branco e colapsados.
Brancos no início/fim	Colapsados (e/ou removidos, conforme spec).
Só brancos (ex.: " ")	Vira 1 espaço (ou vazio).

Conjunto de testes (entrada → saída):

#	Entrada	Saída esperada
T1	" "	" " (vazia)
T2	"abc"	"abc"
T3	"a b c"	"a b c"
T4	"a b" (4 espaços)	"a b"
T5	"a\t\t b"	"a b"
T6	" ab "	" ab " (ou "ab")
T7	" " (só brancos)	" " (ou "")

Inclui-se também o valor-limite de **dois** espaços (menor sequência a ser colapsada), além de testes com tabulações/quebras de linha.

Questão 5 — Teste de regressão e JUnit

Enunciado

O que é teste de regressão? Explique como o uso de testes automatizados e de um framework como o JUnit simplifica os testes de regressão.

Resposta

Teste de regressão é reexecutar testes já existentes após uma mudança (correção, nova funcionalidade, refatoração) para verificar se *nada que funcionava antes foi quebrado*. Manualmente seria caro e repetitivo, então costuma ser negligenciado.

Com automação e **JUnit**:

- Os testes ficam codificados e podem ser executados a qualquer momento com um clique/comando, a custo quase zero.
- Cada teste verifica automaticamente o resultado (**assert**); relatórios indicam imediatamente o que passou/falhou.
- Integram-se à **integração contínua** (CI): toda alteração no repositório dispara a suíte completa, detectando regressões cedo.
- Incentivam refatoração segura — há uma rede de proteção que sinaliza qualquer quebra de comportamento.

Questão 6 — Testar sistema de prateleira adaptado vs. OO

Enunciado

O MHC-PMS é construído adaptando um sistema de informações de prateleira (COTS). Quais as diferenças entre testar esse software e testar um sistema desenvolvido em uma linguagem OO como Java?

Resposta

- **Acesso ao código-fonte:** no COTS, em geral não há fonte; predomina teste *caixa-preta* (funcional), enquanto no sistema Java é possível teste *caixa-branca* (cobertura, unidade).
- **Foco:** no COTS adaptado, testa-se a **configuração**, a integração e as interfaces/extensões; no Java, testam-se classes e métodos individualmente (JUnit) além da integração.
- **Confiança no componente:** parte-se do princípio de que o produto de prateleira já foi testado pelo fornecedor; o esforço concentra-se em verificar se ele atende aos requisitos específicos do MHC-PMS.
- **Ferramentas:** OO permite *mocks*, injeção de dependência e testes automatizados de unidade; no COTS, dependemos de scripts de integração e dados de teste.
- **Defeitos:** no COTS, defeitos no núcleo não podem ser corrigidos (apenas contornados/reportados); no Java, corrige-se o código.

Questão 7 — Cenário de teste da estação no deserto

Enunciado

Escreva um cenário que poderia ser usado para ajudar no projeto de testes do sistema da estação meteorológica no deserto.

Resposta

Cenário — Coleta e transmissão diária sob condições extremas:

“São 12h em um dia de verão no deserto; a temperatura ambiente chega a 50 °C e há tempestade de areia. A estação acorda do modo de economia de energia no horário programado, executa um autoteste dos instrumentos e detecta que o anemômetro está com leitura intermitente devido à areia. Ela coleta os demais dados (temperatura, pressão, umidade, chuva), registra o anemômetro como ‘suspeito’ no relatório de status e arquiva o resumo. Quando o satélite fica visível, a estação tenta transmitir; a primeira tentativa falha por interferência e a estação repete a transmissão até obter confirmação, então volta ao modo de economia de energia.”

Aspectos de teste derivados: limites de temperatura/sensores; autoteste e marcação de instrumento defeituoso; coleta parcial; persistência de dados quando a comunicação falha; *retry* de transmissão; transições de energia; comportamento sob falha de um instrumento sem perder os demais dados.

Questão 8 — Testes de estresse e o MHC-PMS

Enunciado

O que você entende por “testes de estresse”? Sugira como estressar o MHC-PMS.

Resposta

Teste de estresse leva o sistema deliberadamente além de sua carga nominal de projeto para observar como ele se comporta no limite e na falha (degradação suave *vs.* colapso) e se há perda/corrupção de dados. Verifica também a recuperação após sobrecarga.

Como estressar o MHC-PMS:

- Simular **muitos usuários simultâneos** (recepção, enfermeiros, médicos) acima do pico previsto.
- Volume massivo de **registros de pacientes** e consultas ao banco.
- Rajadas de **operações concorrentes** no mesmo prontuário (testar bloqueios/consistência).
- Reduzir recursos (memória/CPU/banda) e observar a degradação.
- Picos no horário de início do expediente (todos logando ao mesmo tempo).
- Verificar se, ao saturar, o sistema **não corrompe dados** e se recupera quando a carga diminui.

Questão 9 — Envolver usuários cedo nos testes de release

Enunciado

Quais as vantagens de envolver os usuários nos testes de release em um estágio inicial? Há desvantagens na participação do usuário?

Resposta

Vantagens:

- Validação realista de requisitos: usuários detectam cedo mal-entendidos e funcionalidades inadequadas, quando corrigir é mais barato.
- Cenários e dados reais de uso, difíceis de antecipar pela equipe.
- Maior aceitação e senso de propriedade do produto final.

Desvantagens:

- Versões iniciais instáveis podem frustrar usuários e gerar má impressão.
- Custo do tempo dos usuários, afastados de suas tarefas.
- *Feedback* pode ser inconsistente, subjetivo ou conflitante.
- Risco de “rolagem de escopo” (novos pedidos) e de testes não sistemáticos.

Questão 10 — Ética de testar até esgotar o orçamento

Enunciado

Discuta a ética de testar um sistema até esgotar o orçamento de teste e, então, entregá-lo a clientes externos.

Resposta

Parar de testar quando o *orçamento* acaba — e não quando um **nível de qualidade/risco** aceitável é atingido — é eticamente problemático, pois desvincula a decisão de entrega da segurança e dos interesses do cliente.

- **Risco ao cliente/usuário:** em sistemas críticos (saúde, finanças, segurança), entregar com qualidade desconhecida pode causar dano — viola o dever de não maleficência e os códigos de ética profissional (ACM/IEEE).
- **Transparência:** se há defeitos conhecidos ou cobertura insuficiente, isso deve ser comunicado ao cliente, que assume o risco de forma informada.
- **Responsabilidade:** o critério de parada deveria ser baseado em risco (severidade, criticidade, cobertura), não puramente orçamentário.

Conclusão: é aceitável encerrar testes quando o risco residual é baixo e acordado; é antiético entregar às cegas, apenas porque o dinheiro acabou, especialmente para clientes externos que confiam na devida diligência do fornecedor.

3 Parte III — Exercise 10 (Stuttgart): Implementação

Observação

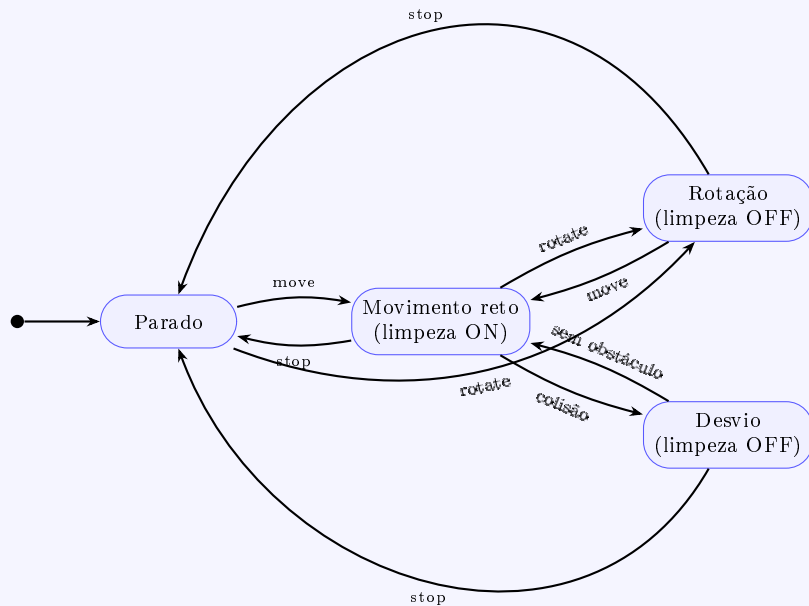
Idiom *switch/while* para *state charts*. Um diagrama de estados pode ser mapeado para código estruturado por uma variável `currentState` e um laço `while(true)` contendo um `switch/case` com um ramo por estado. Em cada `case`: executam-se as *entry actions*; um `while` interno repete as *do actions* enquanto nenhum evento de saída ocorre; ao sair, executam-se as *exit actions* e atualiza-se `currentState` para o estado seguinte (implementando a transição). Estados hierárquicos viram blocos `switch/while` aninhados.

Questão 10.1 — Diagrama de estados do robô de limpeza

Enunciado

Represente, em notação UML, o comportamento do robô de limpeza: move (frente/trás), rotate (gira parado), stop; ao colidir inicia desvio até não haver obstáculo (ou stop); a unidade de limpeza fica ativa no movimento reto e desligada em rotação e desvio.

Resposta



A *entry action* de **Movimento reto** liga a unidade de limpeza; as de **Rotação** e **Desvio** a desligam.

Questão 10.2 — Pseudocódigo (laços while)

Enunciado

Transforme o diagrama de estados em uma implementação. Use pseudocódigo e realize os estados com laços **while**.

Resposta

```
currentState := PARADO
WHILE TRUE
  SWITCH currentState
  CASE PARADO:
    desligaMotores() // entry
    WHILE NOT (cmd=move OR cmd=rotate) DO esperaComando() ENDWHILE
    IF cmd = move THEN currentState := MOV_RETO
    ELSE currentState := ROTACAO
    BREAK
  CASE MOV_RETO:
    ligaLimpeza(); ativaMotoresReto() // entry
    WHILE NOT (cmd=stop OR cmd=rotate OR colisao) DO mover() ENDWHILE
    desligaLimpeza() // exit
    IF colisao THEN currentState := DESVIO
    ELSE IF cmd = rotate THEN currentState := ROTACAO
    ELSE currentState := PARADO
    BREAK
  CASE ROTACAO:
    desligaLimpeza(); ativaMotoresGiro() // entry
    WHILE NOT (cmd=stop OR cmd=move) DO girar() ENDWHILE
    IF cmd = move THEN currentState := MOV_RETO
```

```

ELSE                currentState := PARADO
BREAK
CASE DESVIO:
  desligarLimpeza()           // entry
  WHILE (obstaculo AND cmd<>stop) DO contornar() ENDWHILE
  IF cmd = stop THEN currentState := PARADO
  ELSE                      currentState := MOV_RETO    // sem obstaculo: retoma reto
  BREAK
ENDSWITCH
ENDWHILE

```

Questão 10.3 — Programação estruturada (eliminar GOTO)

Enunciado

Verifique se o fluxograma da Figura 1 é livre de GOTO. Se não, converta-o de forma apropriada e prove que a modificação elimina a necessidade de GOTOs.

Resposta

Critério. Um fluxograma é *livre de GOTO* (bem estruturado) se pode ser construído apenas com **sequência**, **seleção** (if/else) e **iteração** (while) — blocos de *uma* entrada e *uma* saída, adequadamente aninhados.

Diagnóstico. O fluxograma dado *não* é livre de GOTO: assumindo as transições da Figura 1 ($A \rightarrow B/C$; $B \rightarrow D/C$; $C \rightarrow D$; $D \rightarrow C$ ou $D \rightarrow E/\text{fim}$), o nó **D** é alcançado por dois caminhos diferentes (de B e de C) e participa de um laço com C. Isso forma um **laço de múltiplas entradas**, que não pode ser expresso por construções aninhadas simples — exigiria um GOTO.

Conversão (teorema de Böhm–Jacopini). Todo fluxograma pode ser tornado livre de GOTO. Aplica-se o método da **variável de estado** (*switch/while*), que serializa os nós sem saltos:

```

estado := "A"
WHILE estado <> "FIM" DO
  SWITCH estado
    CASE "A": faz_A(); IF a THEN estado:="B" ELSE estado:="C"
    CASE "B": faz_B(); IF b THEN estado:="D" ELSE estado:="C"
    CASE "C": faz_C(); estado:="D"
    CASE "D": faz_D(); IF d THEN estado:="C"
                      ELSE IF e THEN estado:="E" ELSE estado:="FIM"
    CASE "E": faz_E(); estado:="FIM"
  ENDSWITCH
ENDWHILE

```

Prova de que elimina os GOTOs.

- O programa resultante usa *apenas* while, switch e if/else — não há nenhum GOTO. O laço externo tem uma única entrada e uma única saída (`estado="FIM"`).
- **Equivalência:** cada transição do fluxograma original (aresta $X \rightarrow Y$) é reproduzida por uma única atribuição `estado := "Y"`. Assim, qualquer execução do fluxograma corresponde, passo a passo, a uma execução deste laço, e vice-versa — a função computada é a mesma.
- Os antigos saltos (incluindo a entrada “de fora” em D e o retorno $D \rightarrow C$) viram simples mudanças da variável `estado`, resolvendo o problema do laço de múltiplas entradas.

Alternativa mais enxuta: *duplicação de nó* — criar uma cópia **D'** para o caminho vindo de B elimina a aresta que cruzava o laço C–D, deixando o restante exprimível como while aninhado, também sem GOTO.

4 Parte IV — Exercise 11 (Stuttgart): Teste

Questão 11.1 — Objeto de teste, driver e caso de teste

Enunciado

Indique quais partes do programa pertencem ao objeto de teste, ao *driver* de teste e aos casos de teste, explicando o propósito de cada um.

```
int getASCIICode (char character) // retorna o codigo ASCII do caractere
{ ... }
void main () {                    // rotina principal
    ...
    scanf("%c", &characterEnteredByUser); // le entrada do console
    printf("The ASCII code of %c", characterEnteredByUser);
    printf("is 0x%x", getASCIICode(characterEnteredByUser));
    ...
}
```

Resposta

Componente	Descrição
Objeto de teste	A função <code>getASCIICode(char)</code> — a unidade cujo comportamento se quer verificar (<i>unit under test</i>).
Driver de teste	A rotina <code>main()</code> — cria o ambiente para executar o objeto de teste: obtém a entrada (<code>scanf</code>), chama a função e exibe o resultado (<code>printf</code>).
Caso de teste	Cada par concreto <i>entrada</i> → <i>saída esperada</i> , p. ex. entrada 'A' com saída esperada 0x41. O valor lido por <code>scanf</code> fornece a entrada do caso; a saída observada é comparada à esperada.

Propósitos: o *objeto de teste* é o alvo; o *driver* simula o restante do sistema para exercitá-lo de forma isolada; os *casos de teste* definem entradas e os resultados esperados que decidem se o teste passa ou falha.

Questão 11.2 — Medição de cobertura (binário → decimal)

Enunciado

Para o programa que converte um número binário (bit menos significativo primeiro) em decimal: (a) crie o grafo de fluxo de controle; (b) casos de teste para cobertura de instruções (C0); (c) para cobertura de ramos (C1); (d) que característica ainda não foi coberta? (e) por que a cobertura de caminhos (C2) é impossível?

Resposta

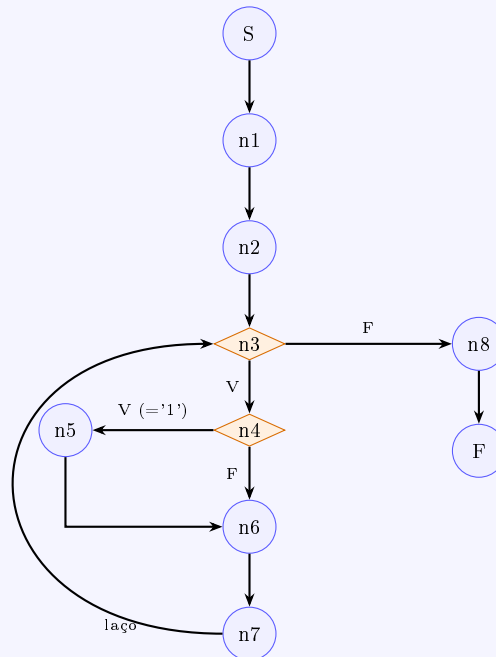
Programa:

```
int convertDecimalNumber() {
    int DecimalNumber = 0; int PowerValue = 0; char Character; // n1
    cin >> Character; // n2
    while ( ((Character=='0')||(Character=='1')) // n3
        && (DecimalNumber < INT_MAX) ) {
        if (Character == '1') // n4
            DecimalNumber = DecimalNumber + pow(2,PowerValue); // n5
        PowerValue++; // n6
        cin >> Character; // n7
    }
    return DecimalNumber; // n8
}
```

```
}

```

(a) Grafo de fluxo de controle:



(b) **Cobertura de instruções (C0):** basta um caso que execute todos os nós. Entrada "1" seguida de um caractere não-binário (terminador), p. ex. '1' e depois 'x': percorre n1, n2, n3(V), n4(V), n5, n6, n7, n3(F), n8. **Conjunto:** { "1x" }.

(c) **Cobertura de ramos (C1):** cada decisão deve assumir V e F. Entrada "1 0 x" (bits 1, 0, terminador):

- n3: V (com '1' e '0') e F (com 'x');
- n4: V (com '1', executa n5) e F (com '0', pula n5).

Conjunto: { "10x" } (cobre todos os ramos; opcionalmente "x" para a saída imediata do laço).

(d) **Característica ainda não coberta:** a segunda condição do `while`, `DecimalNumber < INT_MAX`, nunca se torna *falsa* nos casos acima — ou seja, o **término por estouro (overflow)** e a avaliação da sub-condição de `INT_MAX` permanecem não exercitados (exigiriam uma entrada enorme que aproximasse `DecimalNumber` de `INT_MAX`).

(e) **Por que C2 (cobertura de caminhos) é impossível:** o laço `while` pode repetir um número *arbitrário* de vezes, conforme o comprimento da entrada. Logo, o número de caminhos distintos é **ilimitado** (infinito) e não pode ser totalmente coberto. *Exemplo:* as entradas "x", "0x", "00x", "000x", ... produzem 0, 1, 2, 3, ... iterações — cada uma é um caminho diferente, sem fim.

Referências: Ian Sommerville, **Software Engineering**, 10^a ed., Pearson, 2016 (Cap. 7 — *Design and Implementation*; Cap. 8 — *Software Testing*). IAS — Institute of Industrial Automation and Software Engineering, Universität Stuttgart, **Exercises Software Engineering for Real-Time Systems:** Exercise 10 (Implementation) e Exercise 11 (Test), WS 04/05.